



Research Intake 2026

Day 24

Semantic Analysis & Text Similarity

A Beginner's Guide for Students

Foundational Concepts to Research-Level Understanding

Gudsky Research Foundation

Table of Contents

1	Semantic vs. Syntactic Similarity	4
1.1	Surface-Form Similarity Methods	4
1.2	The Spectrum from Syntactic to Semantic	4
1.3	Semantic Similarity vs. Semantic Relatedness	4
1.4	Why This Module Matters: From Day 22 to Day 24	5
2	WordNet-Based Semantic Similarity	6
2.1	Path-Based Measures	6
2.2	Information-Content (IC) Measures.....	6
2.3	Limitations of WordNet-Based Similarity	7
2.4	Extending to Sentence Level.....	7
3	Embedding-Based Similarity Metrics	8
3.1	Cosine, Euclidean, and Manhattan Distances	8
3.2	Word Mover's Distance (WMD).....	8
3.3	Sentence Embeddings: SBERT	9
3.4	Choosing a Similarity Metric in Practice	9
3.5	Embedding Dimensionality and Compression.....	10
4	Natural Language Inference (NLI)	11
4.1	SNLI and MultiNLI Datasets.....	11
4.2	NLI as a Test of Language Understanding	11
4.3	Zero-Shot Text Classification via NLI.....	12
4.4	NLI as Training Signal for Sentence Embeddings.....	12
5	Semantic Textual Similarity (STS) Benchmarks	13
5.1	STS-B and the SemEval STS Tasks	13
5.2	SBERT and the Sentence Embeddings Revolution	13
5.3	SimCSE: Contrastive Learning for Sentence Embeddings	13
5.4	The Assignment Connection: Computing Spearman on STS-B	14
5.5	STS in Production: Deduplication and Semantic Clustering	14
6	Paraphrase Detection	15
6.1	Microsoft Research Paraphrase Corpus (MRPC)	15
6.2	Types of Paraphrase	15
6.3	Models for Paraphrase Detection	15
6.4	Complete MRPC Training and Evaluation	15
7	Duplicate Question Detection	17
7.1	Quora Question Pairs (QQP).....	17
7.2	Siamese Networks for Question Matching	17

7.3	Beyond Binary: Clustering and Deduplication at Scale.....	17
7.4	Evaluation Metrics for Duplicate Detection	18
8	Semantic Search Architectures	19
8.1	Bi-Encoder vs. Cross-Encoder.....	19
8.2	Re-Ranking Pipelines.....	19
8.3	Dense Passage Retrieval (DPR).....	20
8.4	Hybrid Search: Dense + Sparse	20
8.5	Document Chunking for Retrieval.....	20
9	Applications: Legal, Medical, e-Commerce	22
9.1	Legal Document Semantic Similarity	22
9.2	Medical and Clinical NLP.....	22
9.3	e-Commerce Product Matching	22
9.4	Application Selection Guide	23
9.5	Multilingual Semantic Search.....	23
10	Research: Faithful Similarity and Beyond.....	25
10.1	Advances in Contrastive Sentence Embedding.....	25
10.2	Faithful Similarity: Beyond Distributional Proximity	25
10.3	RAG: Retrieval-Augmented Generation.....	25
10.3b	BEIR: Evaluation at Scale Across Domains	26
10.4	Adversarial Robustness of Semantic Similarity Systems	26
10.5	Summary: Semantic Similarity Methods Landscape	27
A	Assignment: Semantic Search & Text Similarity Pipeline	28
A.1	Task Specifications	28
A.2	Grading Rubric.....	28
A.3	Implementation Tips	28
A.4	Common Pitfalls	29
A.5	Submission Checklist.....	29

1 Semantic vs. Syntactic Similarity

Two sentences can be syntactically dissimilar yet semantically equivalent. 'The patient has a fever' and 'The individual presents with elevated body temperature' share no words but identical meaning. Semantic similarity measurement underpins duplicate detection, question answering, legal document comparison, and the retrieval component of RAG systems.

Surface-form methods count shared characters, tokens, or n-grams — measuring overlap in expression rather than overlap in meaning. Two sentences that express the same idea in different words score low on all surface-form metrics; two sentences that share many words but mean different things can score high. For applications where meaning rather than wording is what matters, semantic similarity is required.

1.1 Surface-Form Similarity Methods

Before neural embeddings made semantic similarity practical at scale, surface-form measures were the standard tools — and remain useful as fast, interpretable baselines:

- **Jaccard similarity:** Ratio of token-set intersection to union. Fast and interpretable, but blind to meaning. Jaccard('the cat sat', 'the dog sat') = $2/4 = 0.5$.
- **Edit distance (Levenshtein):** Minimum character-level insertions, deletions, or substitutions to transform one string into another. Useful for spell correction and near-duplicate detection; not meaningful for semantic comparison.
- **N-gram overlap (BLEU, ROUGE):** Fraction of n-grams shared between two texts. Used in machine translation (BLEU) and summarisation (ROUGE, Day 21). Blind to paraphrase just like Jaccard.
- **BM25:** Probabilistic term-frequency ranking function. Weights rare terms more heavily than common ones. The standard keyword-search baseline in information retrieval. Matches by shared vocabulary, not meaning — motivating the semantic search systems in Section 8.

1.2 The Spectrum from Syntactic to Semantic

The contrast between syntactic and semantic similarity is a spectrum, not a binary. Consider: 'The server crashed.' / 'The system went down unexpectedly.' / 'An outage occurred on the backend infrastructure.' Syntactically very different; lexically almost non-overlapping; semantically nearly identical. A keyword-matching system cannot relate them; a semantic embedding system should cluster them together.

Conversely, 'The car stopped at the light' and 'The light stopped the car' share identical vocabulary and near-identical syntax yet have different semantic relationships — agent-patient role reversal. Bag-of-words similarity would incorrectly judge these as nearly identical; well-trained contextual embeddings would produce distinct representations for the two sentences.

This module (Sections 1–10) covers the full range of techniques: knowledge-graph-based measures (Section 2), embedding-space geometric measures (Section 3), inference-based measures (Section 4), benchmark evaluation (Section 5), and their application to paraphrase detection (Section 6), duplicate question detection (Section 7), semantic search (Section 8), and real-world applications (Section 9).

1.3 Semantic Similarity vs. Semantic Relatedness

'Car' and 'automobile' are highly similar (near-synonyms) and also highly related. 'Car' and 'road' are highly related (associated, co-occurring) but not similar. 'Car' and 'freedom' are moderately related (cultural association) but not similar at all.

Many NLP applications care about similarity (finding semantically equivalent texts). Others care about relatedness (finding topically associated texts even if they express different meanings). A QA system retrieving passages needs relatedness as much as similarity: a passage about 'road conditions affecting vehicle travel times' is relevant to 'how long does it take to drive?' even though the two are not strictly similar. This distinction motivates the different retrieval paradigms in Section 8.

Method	Captures Meaning	Handles Paraphrase	Speed	Requires Training
Jaccard / Edit Distance	No	No	Very fast	No
BM25	Partially (via IDF)	No	Fast	No (corpus statistics)
WordNet similarity	Yes (lexical)	Partially	Moderate	No (uses WordNet)
WMD (word embeddings)	Yes	Yes	Slow (LP)	Embeddings only
SBERT cosine	Yes	Yes	Fast (pre-computed)	Yes (fine-tuning)

1.4 Why This Module Matters: From Day 22 to Day 24

Days 22 and 23 covered sentiment analysis at the sentence and aspect level. Day 24 completes the module by addressing a related but distinct question: not 'what sentiment does this text express?' but 'how similar in meaning are these two texts, and how do we find the most relevant text in a large corpus?'. The two question types are deeply connected in production systems: a customer feedback analytics platform (Day 23, Section 7.1) needs both ABSA to extract aspect-level sentiment from individual reviews AND semantic search to retrieve the most relevant historical feedback in response to a new query.

The progression also connects to earlier modules. The BERT encoder used for SBERT (Section 3.3) is the same architecture studied in depth in Day 19. The NLI training data (Section 4.1) is part of the GLUE benchmark first introduced in Day 19, Section 8. The RAG architecture (Section 10.3) is the downstream application that motivated SBERT's design — and RAG itself underpins many of the production LLM applications discussed throughout the Transformers & LLMs module. Day 24 is deliberately positioned as the module's applied bridge between the earlier NLP building blocks and the retrieval-augmented generation systems that dominate production LLM deployment today.

2 WordNet-Based Semantic Similarity

WordNet (Miller, 1995) organises English words into synonym sets (synsets) connected by semantic relations — hypernymy (is-a), hyponymy, meronymy (part-of), and antonymy. Each synset represents a distinct concept: 'bank' has one synset for the financial institution and another for a river bank. Because WordNet encodes explicit semantic relationships in a graph, it enables similarity measures that genuinely reflect semantic proximity. WordNet-based measures were the dominant approach to word-level semantic similarity for roughly two decades before neural word embeddings. They remain valuable where interpretability is required, where pre-trained embeddings are unavailable for a specialised domain, or in hybrid systems that combine structured knowledge with neural representations.

2.1 Path-Based Measures

Path similarity counts the shortest path between two synsets in the hypernym graph: $\text{sim_path}(s1, s2) = 1 / (\text{path_length} + 1)$, in $[0,1]$. Two identical synsets have path 0, giving similarity 1.0.

Leacock-Chodorow (LCH) normalises by taxonomy depth: $\text{sim_LCH} = -\log(\text{path_length} / (2 \times \text{max_depth}))$, addressing a key weakness of raw path distance: an edge near the top of the hierarchy (connecting broad concepts like 'entity' and 'object') represents a much larger semantic gap than an edge near the bottom (connecting 'spaniel' and 'poodle'), yet both count as a single edge in raw path distance.

Wu-Palmer (WUP), the most widely used path-based measure, uses the depth of the Least Common Subsumer (LCS — the deepest shared ancestor synset): $\text{sim_WUP}(s1, s2) = (2 \times \text{depth}(\text{LCS})) / (\text{depth}(s1) + \text{depth}(s2))$. Wu-Palmer handles depth heterogeneity better than LCH by focusing on LCS depth.

```
# WordNet similarity using NLTK
import nltk
from nltk.corpus import wordnet as wn
nltk.download('wordnet', quiet=True)

dog = wn.synset('dog.n.01')
wolf = wn.synset('wolf.n.01')
car = wn.synset('car.n.01')

print(dog.path_similarity(wolf)) # 0.333 (moderate path similarity)
print(dog.path_similarity(car)) # 0.077 (low)
print(dog.wup_similarity(wolf)) # 0.933 (high: both canids)
print(dog.wup_similarity(car)) # 0.267 (low: unrelated categories)
```

2.2 Information-Content (IC) Measures

Path-based measures treat all edges as equal. IC measures weight concepts by how informative they are: $\text{IC}(c) = -\log(P(c))$, where $P(c)$ is the corpus probability of encountering concept c . Common, high-level concepts ('entity', 'object') have low IC; rare, specific ones ('spaniel', 'metatarsalgia') have high IC.

- **Resnik (1995):** $\text{sim_Resnik}(s1,s2) = \text{IC}(\text{LCS})$. Similar concepts share a more specific (higher IC) LCS. Unbounded (range $[0, \text{max IC}]$).
- **Lin (1998):** $\text{sim_Lin}(s1,s2) = 2 \times \text{IC}(\text{LCS}) / (\text{IC}(s1) + \text{IC}(s2))$. Normalises to $[0,1]$; more comparable across pairs.

- **Jiang-Conrath (1997):** $\text{dist_JCN} = \text{IC}(s1) + \text{IC}(s2) - 2 \times \text{IC}(\text{LCS})$; $\text{sim_JCN} = 1/(\text{dist_JCN} + 1)$. Penalises pairs whose inputs have high IC but share only a very general ancestor.

2.3 Limitations of WordNet-Based Similarity

- **Vocabulary coverage:** ~155,000 English word senses. Proper nouns, technical jargon, neologisms, and multi-word expressions cannot be compared using WordNet measures.
- **Sentence-level extension is non-trivial:** Aggregating word-level scores requires assumptions (averaging, max, IC-weighted) that all introduce failure modes.
- **Language coverage:** IndoWordNet covers 17 Indian languages but at varying completeness; most other WordNets are smaller and less maintained than English WordNet.
- **Static knowledge:** Manually maintained; does not automatically update to reflect new concepts, evolving meanings, or domain-specific terminology.

2.4 Extending to Sentence Level

Three aggregation strategies are commonly used when extending word-level WordNet similarity to sentences:

- **Average pairwise similarity:** For each content word in sentence A, find the best-matching word in sentence B by WordNet similarity, then average the per-word scores. Fast; can be dominated by one highly-similar word pair.
- **IC-weighted average:** Weight each word's contribution by its information content. Prevents common words from inflating the score.
- **Hybrid approaches:** Combine WordNet similarity with syntactic structure and bag-of-words overlap. Pre-neural STS systems using these hybrids achieved Pearson correlations of 0.7–0.8 on SemEval STS 2013–2016 — competitive with simple word-embedding averages, and requiring no labelled training data.

Measure	Range	Strengths	Limitations
Path similarity	[0,1]	Simple, fast	Treats all edges equally
Wu-Palmer (WUP)	[0,1]	Widely used; handles depth variation	No corpus statistics
Resnik (IC)	[0,∞)	Corpus-grounded	Unbounded; word-level only
Lin (IC)	[0,1]	Normalised; comparable	Needs corpus; word-level only
JCN (IC)	[0,1] (as sim)	Penalises specific-to-general pairs	Most complex; word-level only

3 Embedding-Based Similarity Metrics

Neural embeddings represent text as dense vectors in a high-dimensional space, enabling similarity to be measured as geometric proximity. Where WordNet-based measures require a manually maintained knowledge graph, embedding-based measures learn the geometry of semantic space from large text corpora, capturing associations and analogies that no hand-crafted lexical resource explicitly encodes.

3.1 Cosine, Euclidean, and Manhattan Distances

- **Cosine similarity:** $\text{cos_sim}(u,v) = (u \cdot v) / (\|u\| \times \|v\|)$. Measures the angle between vectors, invariant to their magnitude. The default metric for sentence embeddings — used by SBERT, OpenAI embedding APIs, and virtually every semantic search system.
- **Euclidean distance:** $d_{\text{euclid}}(u,v) = \|u-v\|_2$. Sensitive to vector magnitude. For unit-normalised vectors, monotonically related to cosine: $d_{\text{euclid}} = \sqrt{2 \times (1 - \text{cos_sim})}$, so either metric produces the same ranking when vectors are normalised.
- **Manhattan (L1) distance:** $\sum_i |u_i - v_i|$. Robust to outlier dimensions. Used as the ground cost in Word Mover's Distance (Section 3.2).
- **Dot product:** $u \cdot v = \sum_i u_i v_i$. Combines direction and magnitude. Used in DPR (Dense Passage Retrieval) where query-document relevance is modelled as a dot product by design.

```
import numpy as np

def cosine_sim(u, v):
    return np.dot(u, v) / (np.linalg.norm(u) * np.linalg.norm(v))

def euclidean_dist(u, v):
    return np.linalg.norm(u - v)

# For unit-normalised vectors:
# euclidean_dist(u, v) = sqrt(2 * (1 - cosine_sim(u, v)))
# Both metrics give the same candidate ranking
```

3.2 Word Mover's Distance (WMD)

WMD (Kusner et al., 2015) computes the minimum cost of 'transporting' the words of one document to the words of another, using pre-trained word embeddings as the ground cost. It is grounded in the Earth Mover's Distance (Wasserstein distance) from optimal transport theory. Each document is represented as a normalised bag-of-words distribution. The cost of transporting mass from word i in document A to word j in document B is the Euclidean distance between their word embedding vectors. WMD finds the optimal transport plan — minimising total transport cost — and reports that minimum cost as the distance. WMD is effective for short texts where TF-IDF representations are sparse. Its principal limitation is computational cost: exact WMD is $O(p^3 \log p)$ where p is vocabulary size. Relaxed variants (RWMD, Word Centroid Distance) reduce this to $O(p)$. For most production use cases at scale, sentence-level embeddings (SBERT) are more practical.

```
# Word Mover's Distance using gensim
from gensim.models import KeyedVectors
```



```

wv = KeyedVectors.load_word2vec_format('glove.6B.300d.bin', binary=True)

s1 = 'the patient has a high fever'.split()
s2 = 'the individual presents with elevated body temperature'.split()
s3 = 'the stock market crashed yesterday'.split()

print(f'WMD(medical pair): {wv.wmdistance(s1,s2):.3f}') # low
print(f'WMD(unrelated): {wv.wmdistance(s1,s3):.3f}') # high

```

3.3 Sentence Embeddings: SBERT

SBERT (Sentence-BERT, Reimers & Gurevych, 2019) fine-tunes a BERT encoder in a Siamese network architecture on NLI and STS training data, producing sentence embeddings comparable directly via cosine similarity. Standard BERT requires a joint forward pass over both sentences ($O(n^2)$ for all pairs); SBERT's independent encoding reduces pairwise comparison to $O(n)$, enabling semantic search over millions of documents.

```

from sentence_transformers import SentenceTransformer, util

model = SentenceTransformer('all-MiniLM-L6-v2')

sentences = [
    'The patient has a fever.',
    'The individual presents with elevated body temperature.',
    'The stock market dropped significantly.',
]
embeddings = model.encode(sentences, convert_to_tensor=True)
cos_scores = util.cos_sim(embeddings, embeddings)
# [0,1] pair ~ 0.85+ | [0,2] and [1,2] ~ 0.15

```

3.4 Choosing a Similarity Metric in Practice

The right metric depends on the task and the embedding model's normalisation convention:

- **Always use cosine for SBERT and most transformer-based models:** These models are trained with contrastive or NLI objectives that optimise the direction, not magnitude, of the embedding. Cosine correctly ignores magnitude differences.
- **Use dot product for DPR and retrieval-trained models:** DPR training explicitly maximises the dot product between positive query-passage pairs; using cosine on these models loses the information encoded in vector magnitude.
- **Use Euclidean for dimensionality-reduction visualisations:** When projecting high-dimensional embeddings to 2D (e.g., UMAP, t-SNE for cluster analysis), Euclidean distance in the projected space is the natural geometric measure.
- **Consider WMD for short texts without a GPU:** For tasks like short customer feedback comparison or FAQ matching in resource-constrained environments, WMD with GloVe requires no GPU and no fine-tuning.

3.5 Embedding Dimensionality and Compression

Standard SBERT models produce 384-dimensional (MiniLM) or 768-dimensional (BERT-Base) embeddings. At very large scale (hundreds of millions of documents), the memory cost of storing these embeddings can become significant: $100\text{M documents} \times 768 \text{ dimensions} \times 4 \text{ bytes} = \sim 290\text{GB}$. Two strategies manage this:

- **Dimensionality reduction (PCA / Matryoshka):** Matryoshka Representation Learning (Kusupati et al., 2022) trains embeddings where the first d dimensions are already a high-quality d -dimensional representation — so you can truncate to 128 or 64 dimensions at inference time with a controlled quality degradation. E5-large-v2 and several BGE models support Matryoshka representations, making them practical for memory-constrained retrieval.
- **Product quantisation (FAISS IndexIVFPQ):** FAISS's IndexIVFPQ compresses each embedding vector to a fixed number of bytes (typically 8–64) using learned codebooks. At 100M documents, IndexIVFPQ with 32 bytes per vector reduces storage from $\sim 290\text{GB}$ to $\sim 3.2\text{GB}$ with only a small accuracy degradation in $\text{recall}@10$.

4 Natural Language Inference (NLI)

Natural Language Inference (NLI), also called Recognising Textual Entailment (RTE), determines the logical relationship between a premise (P) and a hypothesis (H): Entailment (P being true guarantees H is true), Contradiction (P being true means H must be false), or Neutral (P and H can both be true independently). NLI captures directional meaning relationships that symmetric similarity scores miss: if P entails H but H does not entail P, the two texts are semantically related but not interchangeable. NLI serves two roles: as a training task teaching models deep semantic reasoning (making them better at downstream tasks requiring inference), and as a zero-shot classification engine (framing classification as entailment). Both are covered here.

4.1 SNLI and MultiNLI Datasets

SNLI (Stanford NLI, Bowman et al., 2015) contains 570,000 sentence pairs. Premises came from Flickr30k image captions; Mechanical Turk workers wrote Entailment, Contradiction, and Neutral hypotheses for each. This produced a large, diverse dataset but with known annotator biases: contradictions tended to add negation words; neutral examples used 'maybe' or 'possibly', creating surface patterns that models can exploit without genuine reasoning. MultiNLI (Williams et al., 2018) extended SNLI across ten genres (government documents, fiction, telephone conversations, travel guides) and introduced a matched/mismatched evaluation split — the mismatched set contains genres absent from training, testing cross-genre generalisation. MultiNLI is included in GLUE; BERT-Large achieved 86.7% accuracy on the matched test set, a landmark result when published.

Dataset	Year	Pairs	Domains	Key Feature
SNLI	2015	570,000	Image captions	First large-scale NLI; well-studied biases
MultiNLI	2018	433,000	10 genres	Cross-genre; used in GLUE
ANLI	2019	170,000	Adversarially constructed	Human-in-loop adversarial collection
FEVER-NLI	2019	185,445	Wikipedia fact-checking	Fact verification; real-world claims
MedNLI	2018	14,049	Clinical notes (MIMIC-III)	Clinical domain; expert annotation

4.2 NLI as a Test of Language Understanding

NLI requires reasoning about lexical entailment ('running' entails 'moving'), quantifier scope ('all dogs barked' entails 'some dogs barked' but not vice versa), negation ('no fever' contradicts 'has a fever'), presupposition ('she stopped smoking' presupposes she smoked), and world knowledge. This breadth makes NLI one of the most diagnostically rich tasks in NLP. Adversarial NLI (ANLI, Nie et al., 2019) addressed the finding that BERT-scale models had essentially saturated SNLI/MultiNLI without genuine reasoning — they were exploiting annotator biases. ANLI was collected via a human-and-model-in-the-loop procedure: annotators wrote hypotheses to fool the current best model; only examples that fooled it were retained. Even strong models achieve only 60–70% on ANLI, compared to ~90% human accuracy.

4.3 Zero-Shot Text Classification via NLI

A powerful application: classify text into user-defined categories without any labelled training data by framing classification as NLI. For each candidate label, construct the hypothesis 'This text is about {label}' and score the entailment probability. The label with the highest entailment score wins.

```
from transformers import pipeline

classifier = pipeline('zero-shot-classification',
                     model='facebook/bart-large-mnli')

text = 'The new iPhone has an exceptional camera but poor battery life.'
labels = ['technology review', 'cooking', 'sports', 'finance']

result = classifier(text, labels)
for label, score in zip(result['labels'], result['scores']):
    print(f'{label}: {score:.3f}')
# 'technology review' scores highest

# Multi-label classification:
result = classifier(text, ['positive camera', 'negative battery'], multi_label=True)
```

Zero-shot NLI classification is valuable where labelled training data is unavailable or costly, and as a complement to embedding-based search when categories can be expressed as natural-language descriptions. It is sensitive to hypothesis template phrasing — 'this text is about X' vs. 'the topic of this text is X' can yield meaningfully different scores — motivating careful prompt engineering in deployment.

4.4 NLI as Training Signal for Sentence Embeddings

One of NLI's most impactful roles in the field is as a training signal for sentence embedding quality. SBERT (Section 3.3) was originally fine-tuned on NLI data using a Siamese architecture: entailment pairs acted as positive pairs (sentences that should be nearby in embedding space), contradiction pairs as hard negatives (sentences that should be far apart despite often sharing vocabulary), and neutral pairs somewhere between. This NLI-trained sentence embedding achieves far higher STS-B Spearman correlation than the naive BERT CLS token (0.869 vs. 0.203), demonstrating that the directional semantic reasoning captured by NLI training transfers cleanly to symmetric similarity ranking. SimCSE (Section 5.3) extended this by using NLI data in an explicit contrastive learning framework: for each anchor sentence, its entailment counterpart is the positive, its contradiction counterpart is the hard negative, and all other sentences in the batch are in-batch negatives. The contrastive InfoNCE loss used in SimCSE pulls the anchor and positive closer while pushing the anchor and all negatives apart — a cleaner, more geometrically motivated objective than the classification-based approach used in the original SBERT. The combination of NLI supervision and contrastive learning remains the foundation of most state-of-the-art sentence embedding training pipelines as of 2025.

5 Semantic Textual Similarity (STS) Benchmarks

STS assigns a continuous similarity score to a sentence pair, reflecting how close the two sentences are in meaning. Unlike NLI (discrete 3-class) or paraphrase detection (binary), STS evaluates a model's ability to produce a graded score that correlates with human judgements. This makes STS the standard evaluation framework for sentence embedding models.

5.1 STS-B and the SemEval STS Tasks

The SemEval STS shared tasks (2012–2016) collected sentence pairs from diverse sources (news headlines, image captions, forum posts, technical descriptions) and had human annotators rate similarity from 0 (completely unrelated) to 5 (completely equivalent). STS-B (Cer et al., 2017) merged the best sentence pairs from multiple SemEval STS tasks into a single 8,628-pair benchmark (5,749 train / 1,500 dev / 1,379 test). STS-B is included in GLUE. Performance is measured by Spearman rank correlation (ρ) between model-predicted scores and human scores. Spearman is preferred over Pearson because it is rank-based (robust to non-linear monotonic relationships) and because the absolute scale of model scores matters less than their relative ordering.

5.2 SBERT and the Sentence Embeddings Revolution

Before SBERT, computing STS with BERT required joint encoding of each pair — $O(n^2)$ forward passes for all pairs in a corpus. SBERT fine-tuned BERT in a Siamese configuration on NLI + STS data, producing independent sentence embeddings comparable via cosine similarity. SBERT achieved Spearman $\rho = 0.869$ on STS-B while reducing inference time from ~65 hours to ~5 seconds for a 10,000-sentence comparison task — a ~47,000 $\times$ speedup that made dense semantic search over large corpora practical for the first time.

Model	STS-B Spearman ρ	Speed	Notes
GloVe average	0.58	Very fast	Simple baseline; no context
InferSent (BiLSTM)	0.771	Fast	Pre-BERT state-of-the-art
BERT CLS token	0.203	Slow (joint)	Surprisingly poor without fine-tuning
BERT avg. token emb.	0.787	Slow (joint)	Better than CLS; still $O(n^2)$
SBERT (NLI + STS)	0.869	Fast (independent)	Standard for sentence similarity
SimCSE (contrastive)	0.921	Fast (independent)	State-of-the-art (2021)

5.3 SimCSE: Contrastive Learning for Sentence Embeddings

SimCSE (Gao et al., 2021) achieves state-of-the-art STS performance with a simple contrastive objective. The unsupervised variant passes the same sentence through the encoder twice with different dropout masks, treating the two embeddings as a positive pair and all other sentences in the batch as negatives. This forces stable, consistent representations while pushing apart representations of different sentences. The supervised variant uses NLI data: entailment pairs as positives, contradiction pairs as hard negatives. Supervised SimCSE achieves $\rho = 0.921$ on STS-B — a 5-point improvement over SBERT. SimCSE established contrastive learning as the dominant paradigm for sentence embedding training, motivating E5, GTE, and BGE — the models underpinning most production semantic search systems as of 2025.

5.4 The Assignment Connection: Computing Spearman on STS-B

Task 1 of this module's assignment (Section A) requires computing Spearman ρ on the STS-B test set. A correct implementation has three steps: load the test pairs and their human scores; encode each sentence independently using the chosen model; compute cosine similarity for each pair; report `scipy.stats.spearmanr` between predicted scores and human scores. The Spearman correlation — not accuracy, not RMSE — is the correct metric because it is rank-based and is what published baselines report.

```
from sentence_transformers import SentenceTransformer, util
from scipy.stats import spearmanr
from datasets import load_dataset

model = SentenceTransformer('all-MiniLM-L6-v2')
stsb = load_dataset('glue', 'stsb', split='test')

s1_emb = model.encode(stsb['sentence1'], normalize_embeddings=True)
s2_emb = model.encode(stsb['sentence2'], normalize_embeddings=True)

# Cosine similarity (dot product of unit vectors = cosine)
import numpy as np
pred_scores = [float(np.dot(a, b)) for a, b in zip(s1_emb, s2_emb)]
human_scores = stsb['label'] # 0-5 scale

rho, pval = spearmanr(pred_scores, human_scores)
print(f'Spearman rho: {rho:.3f}') # expect ~0.85+ for all-MiniLM-L6-v2
```

5.5 STS in Production: Deduplication and Semantic Clustering

Beyond benchmark evaluation, STS scores power several production capabilities that Gudskey team members building document analysis pipelines will encounter:

- **Near-duplicate detection:** Compute pairwise cosine similarity over a corpus of documents; pairs exceeding a threshold (e.g., 0.92) are flagged as near-duplicates for deduplication. At scale, ANN search (Section 8) replaces exhaustive pairwise comparison.
- **Semantic clustering:** Applying k-means or HDBSCAN to sentence embeddings groups semantically related sentences regardless of surface form. This is useful for summarising large collections of customer feedback (grouping complaints about the same issue across different phrasings) or for organising research literature (grouping papers on the same topic).
- **Semantic change detection:** Monitoring how the cosine similarity between the embedding of a topic keyword and embeddings of documents in a corpus changes over time detects semantic drift — the gradual evolution of how a concept is discussed in a domain. This is valuable for tracking evolving customer concerns, policy framing, or scientific terminology.

6 Paraphrase Detection

Paraphrase detection determines whether two sentences express the same meaning using different words: a binary label (paraphrase / not paraphrase) rather than the continuous STS score. Applications include deduplication of user-generated content, FAQ clustering, legal document equivalence, and meaning-level plagiarism detection.

6.1 Microsoft Research Paraphrase Corpus (MRPC)

MRPC (Dolan & Brockett, 2005) contains 5,801 sentence pairs from news articles, annotated as paraphrase (1) or non-paraphrase (0). The corpus is imbalanced (~67% paraphrases) because it was constructed by filtering for similar pairs before manual verification. MRPC is in GLUE; BERT-Base achieves ~88% accuracy and ~92% F1, close to estimated human performance (86% accuracy, 92% F1 based on inter-annotator agreement).

6.2 Types of Paraphrase

- **Lexical substitution:** Synonyms or near-synonyms preserve syntactic structure. 'The car is fast.' / 'The automobile is quick.' Easiest type: synonym-aware embeddings handle it well.
- **Syntactic variation:** Same proposition in different grammatical structure. 'John gave Mary a book.' / 'Mary was given a book by John.' Requires parsing or contextual embeddings.
- **Semantic equivalence via world knowledge:** 'The surgery was performed in the morning.' / 'The operation took place before noon.' Requires knowing surgery = operation, morning $\subseteq$ before noon.
- **Quasi-paraphrase (partial equivalence):** 'The train was delayed by 15 minutes.' / 'The train arrived late.' Semantically overlapping but not identical. Hard cases for both models and human annotators.

6.3 Models for Paraphrase Detection

Modern systems are typically trained as sentence-pair classifiers: BERT or RoBERTa encodes the concatenated pair; a binary classification head on [CLS] predicts the label. Cross-encoders (jointly encoding both sentences) outperform bi-encoders because full attention allows direct comparison of both sentences, catching cases where sentences seem similar out of context but differ on specific claims. Hard negative mining — constructing training negatives that are highly similar in surface form or topic but not paraphrases — substantially improves quality. Models trained on randomly sampled negatives tend to learn surface similarity rather than genuine semantic equivalence. The PAWS dataset (Paraphrase Adversaries from Word Scrambling) provides hard-negative-enriched training data: it contains non-paraphrase pairs with near-identical word overlap, designed to force models to look beyond surface form.

6.4 Complete MRPC Training and Evaluation

The following shows the complete training configuration for Task 2 of the assignment, including the TrainingArguments and evaluation compute:

```
from transformers import (AutoTokenizer, AutoModelForSequenceClassification,
                          TrainingArguments, Trainer)
from datasets import load_dataset
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix
import numpy as np
```

```

tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')
model = AutoModelForSequenceClassification.from_pretrained(
    'bert-base-uncased', num_labels=2)

def tokenize(batch):
    return tokenizer(batch['sentence1'], batch['sentence2'],
                    truncation=True, max_length=128)

ds = load_dataset('glue', 'mrpc')
ds = ds.map(tokenize, batched=True)

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
    return {'accuracy': accuracy_score(labels, preds),
            'f1': f1_score(labels, preds)}

args = TrainingArguments(
    output_dir='./mrpc-bert',
    num_train_epochs=3, learning_rate=2e-5,
    per_device_train_batch_size=16, per_device_eval_batch_size=32,
    eval_strategy='epoch', save_strategy='epoch',
    load_best_model_at_end=True, metric_for_best_model='f1',
    fp16=True,          # enable if GPU is available
    report_to=[],
)
trainer = Trainer(model=model, args=args, train_dataset=ds['train'],
                  eval_dataset=ds['validation'], compute_metrics=compute_metrics)
trainer.train()
results = trainer.evaluate(ds['test']) # report these numbers

```


7 Duplicate Question Detection

Duplicate question detection is a high-volume application of paraphrase detection at Q&A and community knowledge-base platforms. Millions of users asking semantically equivalent questions ('How do I lose weight fast?' / 'What is the quickest way to reduce body weight?') should be linked to the same answer thread. Detecting these duplicates consolidates answers, directs users to existing high-quality threads, and improves search relevance within the knowledge base.

7.1 Quora Question Pairs (QQP)

QQP (Iyer et al., 2017) contains 400,000+ question pairs from Quora, annotated as duplicate (1) or not (0). It is in GLUE, and at this scale it is well-suited for training large neural models despite known annotation noise. Class distribution is ~37% duplicates. BERT-Large achieves ~91.3% accuracy on QQP.

7.2 Siamese Networks for Question Matching

Siamese networks process both questions through the same encoder independently and compare their representations. The critical advantage for production deployment: question embeddings can be pre-computed and indexed once, reducing new-question duplicate detection to a nearest-neighbour lookup — $O(1)$ per query rather than $O(N)$ pairwise classification. Stack Overflow has ~55 million questions. Pre-computing all embeddings and indexing with FAISS enables millisecond-latency duplicate detection at this scale. The bi-encoder retrieval + cross-encoder re-ranking pipeline from Section 8 applies directly: the bi-encoder retrieves top-K similar questions; the cross-encoder verifies whether any are true duplicates.

7.3 Beyond Binary: Clustering and Deduplication at Scale

Production systems do more than classify isolated pairs. The real task is clustering: grouping all questions into equivalence classes, each pointing to a canonical question and its accepted answers. Pairwise-to-clustering creates challenges: transitivity (A duplicate of B and B duplicate of C does not always mean A is a duplicate of C), threshold calibration (where does 'duplicate' end and 'related but distinct' begin?), and online incremental assignment of new questions to existing clusters.

```
from sentence_transformers import SentenceTransformer
import faiss, numpy as np

model = SentenceTransformer('all-MiniLM-L6-v2')

# --- One-time offline step ---
existing_qs = [...] # list of existing questions
existing_embs = model.encode(existing_qs, normalize_embeddings=True)
index = faiss.IndexFlatIP(existing_embs.shape[1])
index.add(existing_embs)

# --- Online: find duplicates for a new question ---
def find_duplicates(new_q, threshold=0.85, top_k=10):
    emb = model.encode([new_q], normalize_embeddings=True)
    scores, ids = index.search(emb, top_k)
    return [(existing_qs[i], float(s))
            for s, i in zip(scores[0], ids[0]) if s >= threshold]
```

7.4 Evaluation Metrics for Duplicate Detection

Duplicate question detection systems are typically evaluated on three metrics, each capturing a different aspect of the detection quality:

- **Precision@K:** Of the top-K questions retrieved as potential duplicates, what fraction are true duplicates? Measures the cost of false alarms (sending users to irrelevant threads). High precision is important when the consequences of a false positive are significant (e.g., closing a legitimate new question as a duplicate).
- **Recall@K:** Of all true duplicates that exist in the database, what fraction appear in the top-K retrieved results? Measures the cost of false negatives (failing to link a question to its existing answers). High recall is important when the priority is ensuring users always find existing answers.
- **Mean Reciprocal Rank (MRR):** For each query, the reciprocal of the rank of the first true duplicate in the retrieved list, averaged across all queries. MRR rewards systems that rank the true duplicate highly even if it is not at position 1 — appropriate when users will inspect the top few results rather than only the top one.

8 Semantic Search Architectures

Semantic search applies semantic similarity to information retrieval: given a natural language query, find the most semantically relevant documents or passages regardless of exact keyword overlap. Semantic search systems are the operational core of RAG pipelines, enterprise knowledge bases, QA systems, and legal/medical document retrieval.

8.1 Bi-Encoder vs. Cross-Encoder

Bi-encoders encode query and document independently and compare via cosine similarity — enabling pre-computation of all document embeddings and millisecond retrieval over millions of documents. Cross-encoders jointly encode (query, document) pairs through full attention — much higher quality but $O(N)$ forward passes per query. Production systems use bi-encoder retrieval followed by cross-encoder re-ranking. The distinction reflects a fundamental precision-speed trade-off. A bi-encoder pre-computes every document embedding offline. At query time, a single query embedding is compared against all document embeddings via approximate nearest-neighbour (ANN) search, returning the top-100 or top-1000 candidates in milliseconds. A cross-encoder requires one full forward pass per candidate, directly modelling lexical overlap, syntactic alignment, and semantic entailment between query and document — at the cost of making exhaustive corpus-level retrieval impractical.

8.2 Re-Ranking Pipelines

The two-stage retrieve-then-rerank pipeline combines the speed of bi-encoder retrieval with the quality of cross-encoder scoring. Stage 1: the bi-encoder retrieves top-K candidates (typically 100–1000) via ANN search. Stage 2: the cross-encoder scores each candidate against the query and re-ranks them. The final top-k (typically 3–10) results are returned to the user or passed to a downstream generator in a RAG pipeline.

```
from sentence_transformers import SentenceTransformer, CrossEncoder
import faiss, numpy as np

bi_enc = SentenceTransformer('all-MiniLM-L6-v2')
cross_enc = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2')

# One-time: build FAISS index
doc_embs = bi_enc.encode(corpus, normalize_embeddings=True)
index = faiss.IndexFlatIP(doc_embs.shape[1])
index.add(doc_embs)

def semantic_search(query, k_retrieve=100, k_return=5):
    q_emb = bi_enc.encode([query], normalize_embeddings=True)
    scores, ids = index.search(q_emb, k_retrieve)
    candidates = [corpus[i] for i in ids[0]]
    ce_scores = cross_enc.predict([[query, doc] for doc in candidates])
    ranked = sorted(zip(ce_scores, candidates), reverse=True)
    return ranked[:k_return]
```

FAISS (Facebook AI Similarity Search) is the standard open-source ANN library, supporting exact (IndexFlatIP) and approximate (IndexHNSWFlat, IndexIVFPQ) search. For corpora up to a few million

documents, exact search is fast enough; at tens of millions, approximate search provides a controllable accuracy-speed trade-off.

8.3 Dense Passage Retrieval (DPR)

DPR (Karpukhin et al., 2020) is a bi-encoder trained specifically for open-domain QA: the question and passage encoders are fine-tuned jointly to maximise the dot product between a question embedding and embeddings of answer-containing passages, using hard negatives mined from BM25. DPR substantially outperforms BM25 on Natural Questions and TriviaQA retrieval benchmarks, and became the retrieval backbone of the original RAG paper (Lewis et al., 2020).

8.4 Hybrid Search: Dense + Sparse

In practice, dense semantic retrieval and sparse lexical retrieval (BM25) are complementary: BM25 reliably finds documents sharing rare, discriminative keywords with the query; dense retrieval finds semantically related documents that may share no keywords. Hybrid search combines both signals, typically via Reciprocal Rank Fusion (RRF): the two ranked lists are combined by summing reciprocal ranks, producing a merged ranking that captures both exact-match and semantic-match relevance.

Component	Method	Latency	Strength	Weakness
Bi-encoder retrieval	SBERT / DPR / E5 + FAISS	~5ms	Fast; scales to millions	Lower precision than cross-encoder
Cross-encoder re-rank	MonoBERT / MiniLM CE	~200ms (100 cands)	High precision	O(N); can't search full corpus
Lexical baseline	BM25 (Elasticsearch)	~5ms	Exact match; interpretable	Misses paraphrase/synonym matches
Hybrid (BM25 + dense)	Reciprocal Rank Fusion	~10ms	Best of both worlds	Requires both indices

8.5 Document Chunking for Retrieval

Most real-world documents are longer than a sentence — paragraphs, pages, or entire reports — yet SBERT and similar models are trained and optimised on sentence-to-paragraph length inputs (typically up to 512 tokens). For documents longer than the model's context window, chunking is required: splitting documents into overlapping or non-overlapping segments before embedding. The chunking strategy significantly affects retrieval quality:

- **Fixed-size chunking:** Split every document into chunks of N tokens with a stride of S tokens (so consecutive chunks overlap by N-S tokens). Simple and fast; overlap ensures that sentences near chunk boundaries are represented in at least one chunk without truncation. Common defaults: N=512, S=256 for BERT-based models.
- **Semantic chunking:** Split at natural sentence or paragraph boundaries rather than fixed token counts, preserving the semantic coherence of each chunk. Semantically coherent chunks tend to produce better embeddings than chunks that cut mid-sentence or mid-paragraph.
- **Hierarchical indexing:** Index documents at multiple levels (sentence, paragraph, section), allowing retrieval to return the most specific relevant passage rather than a full-document embedding. The RAPTOR approach (Sarathi et al., 2024) recursively clusters and summarises leaf

chunks, building a tree of progressively higher-level representations that enables retrieval at any level of granularity.

For the assignment's Task 3, a simple sentence-level or fixed-256-token chunking strategy is sufficient. Production RAG systems typically invest significant engineering effort in chunking strategy, as it is one of the most impactful practical factors in retrieval quality, often larger in effect than the choice of embedding model for well-served domains.

9 Applications: Legal, Medical, e-Commerce

The semantic similarity and search techniques from Sections 1–8 are deployed in production across a wide range of industries. Three domains are highlighted here because they represent the breadth of application requirements: legal document retrieval (precision and interpretability paramount), medical/clinical NLP (accuracy and safety critical), and e-commerce product matching (scale — hundreds of millions of items — the primary constraint).

9.1 Legal Document Semantic Similarity

Legal reasoning depends on precedent: courts and lawyers must find prior cases with factually and legally similar circumstances. Keyword search is limited by extreme variability in legal language: the same legal concept may be expressed as 'duty of care', 'standard of care', 'reasonable care obligation', 'negligence standard', or 'safeguarding duty' across different jurisdictions, eras, and writing styles. Semantic similarity retrieval correctly identifies that all these formulations are semantically related even with no shared keywords.

Legal Document Semantic Similarity

ROSS Intelligence (before GPT era) used SBERT-style embeddings to match legal query briefs against 50 million US case law paragraphs. Semantic retrieval found relevant precedents that keyword search missed because legal language often uses unusual formulations — 'duty of care was breached' matches 'negligence in safeguarding' without sharing a single word.

Legal BERT (Chalkidis et al., 2020) and related domain-adapted models are pre-trained on legal corpora (EU legislation, contracts, US court cases). These outperform general-purpose SBERT on legal IR tasks by 5–10 F1 points, illustrating the domain-adaptive pre-training principle from Day 20, Section 7. Gains are largest for tasks involving older case language and jurisdiction-specific terminology not well-represented in general pre-training data.

9.2 Medical and Clinical NLP

Clinical NLP applications of semantic similarity include: clinical trial matching (finding patients whose diagnoses semantically match trial inclusion/exclusion criteria), medical QA (retrieving relevant clinical evidence for a physician's query), and drug information retrieval (matching patient-described symptoms against structured pharmacological databases). The clinical setting imposes two requirements beyond general semantic search: safety-critical precision (a false positive in trial matching could enrol an ineligible patient; a false negative could exclude one who might benefit) and knowledge-grounded accuracy (UMLS — Unified Medical Language System — and SNOMED CT provide structured medical knowledge that should ground similarity beyond pure embedding geometry). BioBERT, PubMedBERT, and Clinical BERT provide the domain-adapted embeddings most commonly used as backbones of clinical semantic search systems.

9.3 e-Commerce Product Matching

Use Case: e-Commerce Product Deduplication

Amazon's product catalogue has 350 million listings. Semantic duplicate detection using bi-encoder embeddings

identifies listings for identical products from different sellers — consolidating reviews, correcting prices, and improving search relevance. A cross-encoder re-ranker verifies the top-100 bi-encoder candidates, with the full pipeline handling 10M new listings per day.

Product matching requires semantic similarity across multiple modalities (title, description, specifications) and different seller catalogue schemas. A production pipeline combines: a product-specific embedding model fine-tuned on verified duplicate pairs; a structured attribute comparison module for technical specifications; and a final cross-encoder jointly scoring title+description+attribute match. This multi-signal fusion substantially outperforms text-only similarity for products with distinctive numeric specifications (memory, screen size, resolution) where embedding the description may not reliably distinguish '128GB' from '256GB'.

9.4 Application Selection Guide

The following table maps the major application domains to the recommended semantic similarity architecture, the key technical requirement driving that choice, and the primary risk to manage in production:

Application	Recommended Architecture	Key Requirement	Primary Risk
Legal case retrieval	Domain-adapted SBERT (LegalBERT) + FAISS	Semantic match across terminology variants	Missing precedent on novel legal language
Clinical trial matching	BioBERT bi-encoder + cross-encoder verify	Safety-critical precision	False negatives excluding eligible patients
Medical QA	PubMedBERT RAG pipeline	Knowledge-grounded accuracy	Factually inconsistent retrieval
e-Commerce dedup	Bi-encoder + attribute comparison + CE	Scale (100M+ items)	Missing duplicate on spec-different descriptions
Enterprise FAQ / Q&A	SBERT + FAISS + BM25 hybrid	Coverage of exact + semantic queries	Threshold calibration across topic areas

9.5 Multilingual Semantic Search

Building on the multilingual sentiment analysis techniques from Day 23 (Section 7), multilingual semantic search applies the same bi-encoder + FAISS architecture across multiple languages simultaneously. The key challenge: a query in Hindi should retrieve semantically relevant documents in English, Bengali, or Tamil, even with no keyword overlap across languages. Multilingual SBERT models (paraphrase-multilingual-MiniLM-L12-v2, multilingual-e5-large) produce language-agnostic sentence embeddings by fine-tuning a multilingual encoder on parallel translation pairs: the English sentence 'The patient has a fever' and its Hindi translation 'rogi ko bukhari hai' are trained to have the same embedding. A single FAISS index can then serve cross-lingual retrieval: a Hindi query embedding finds English documents in the same index with no language-specific component.

```
# Cross-lingual semantic search with multilingual SBERT
from sentence_transformers import SentenceTransformer
```

```
import faiss, numpy as np

# Multilingual model: same embedding space for 100+ languages
model = SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')

# English corpus
docs = ["The patient has a high fever.",
        "The stock market dropped significantly.",
        "Monsoon rains have been heavy this season."]
doc_embs = model.encode(docs, normalize_embeddings=True)
index = faiss.IndexFlatIP(doc_embs.shape[1])
index.add(doc_embs)

# Hindi query retrieves from English corpus
hindi_query = 'rogi ko bukhar hai' # 'the patient has a fever'
q_emb = model.encode([hindi_query], normalize_embeddings=True)
scores, ids = index.search(q_emb, k=2)
for s, i in zip(scores[0], ids[0]):
    print(f'{s:.3f} {docs[i]}')
# Highest score: 'The patient has a high fever.'
```


10 Research: Faithful Similarity and Beyond

The frontier of semantic similarity research addresses limitations shared by current embedding-based systems: they measure distributional proximity (texts that occur in similar contexts have similar embeddings) but not necessarily factual faithfulness (semantically close texts may differ in specific claims), and they are vulnerable to adversarial examples designed to fool the similarity function. This section covers three active research directions: advances in contrastive sentence embedding, faithful/factual similarity, and the RAG architecture that brought semantic search to the centre of production LLM systems.

10.1 Advances in Contrastive Sentence Embedding

SimCSE (Section 5.3) established contrastive learning as the dominant paradigm for sentence embedding training. Subsequent work has continued to improve on it:

E5 (Wang et al., Microsoft, 2022) trained text embeddings on a massive multi-task mixture of (query, passage) pairs from web data, using a simple 'Query: ...' / 'Passage: ...' prefix to condition the encoder on the retrieval task. E5 achieves state-of-the-art on BEIR (Benchmarking Information Retrieval, a heterogeneous retrieval benchmark across 18 datasets spanning biomedical, legal, financial, and general web domains). The prefix conditioning strategy is simple and highly effective, making E5 a strong general-purpose starting point for fine-tuning on domain-specific retrieval tasks.

BGE (BAAI General Embedding, Xiao et al., 2023) and GTE (Generalised Text Embeddings, Li et al., 2023) follow similar multi-stage pipelines: contrastive pre-training on large text pairs, followed by fine-tuning on curated high-quality pairs with hard negatives. These are currently among the top-performing models on the MTEB (Massive Text Embedding Benchmark, Muennighoff et al., 2022) leaderboard, which evaluates embeddings across 56 tasks including classification, clustering, reranking, retrieval, STS, and summarisation. MTEB provides the most comprehensive current evaluation of sentence embedding quality.

10.2 Faithful Similarity: Beyond Distributional Proximity

Standard similarity measures capture distributional proximity but not factual faithfulness. 'The study found that the treatment was effective' and 'The study found that the treatment was ineffective' differ by only one prefix token, yet are factually contradictory. A high cosine similarity score for this pair would be deeply misleading in a medical information retrieval context.

Faithful similarity research explores embedding spaces and measures sensitive to factual content:

- **Fact-aware pre-training:** Expose the model to factual contradiction examples during contrastive training, using NLI contradiction pairs as explicit hard negatives.
- **NLI-augmented similarity:** Combine embedding cosine similarity with an NLI entailment score. High similarity requires both distributional proximity and logical consistency.
- **Knowledge-grounded similarity:** Ground comparison in a structured knowledge base (Section 8.3's commonsense knowledge integration from Day 23) rather than relying solely on distributional representations.

This research direction connects directly to the hallucination problem in LLM generation (Day 21, Section 8): a RAG system that retrieves factually inconsistent passages provides the generator with contradictory grounding, a major cause of LLM hallucination. The quality of faithful retrieval directly determines the quality of grounded generation.

10.3 RAG: Retrieval-Augmented Generation

Research Spotlight: Sentence Embedding for Retrieval-Augmented Generation

Lewis et al. (2020) combined dense retrieval (bi-encoder FAISS index) with seq2seq generation (BART) to produce the RAG architecture. The retriever fetches relevant document chunks; the generator conditions on both query and retrieved passages. RAG became the architectural foundation for production LLM Q&A systems at Google, Microsoft, and Meta.

RAG applies the bi-encoder + re-ranker architecture of Section 8 as the retrieval component of a generative system. The retriever (DPR, E5, or a domain-adapted model) maintains a FAISS index; at inference time, the user's query retrieves the top-K most relevant passages; those passages are concatenated with the query and fed to a generative model (BART, T5, or a GPT-family model) which produces the final answer conditioned on both query and retrieved context. RAG's key advantage over purely parametric LLMs: the knowledge base can be updated without retraining the model — adding a new document simply requires computing and indexing its embedding. This makes RAG substantially more maintainable than fine-tuning for knowledge-intensive applications requiring up-to-date factual grounding. The quality of the semantic search component directly determines the quality of generated answers: a retriever that misses the relevant passage or retrieves factually inconsistent passages cannot be corrected by even the most capable generator.

10.3b BEIR: Evaluation at Scale Across Domains

The BEIR benchmark (Thakur et al., 2021) provides a comprehensive evaluation of zero-shot information retrieval generalisation across 18 heterogeneous datasets: biomedical (TREC-COVID, NFCorpus, BioASQ), scientific (SCIFACT, ArguAna), legal (FIQA), news (TREC-NEWS, Robust04), e-commerce (TREC-Product), and others. A model is trained on MS MARCO (a general web QA dataset) and evaluated zero-shot on all 18 datasets, measuring nDCG@10 (Normalised Discounted Cumulative Gain at rank 10).

BEIR revealed a critical finding: models that achieve state-of-the-art on in-domain benchmarks like MS MARCO often show substantial performance drops on out-of-domain datasets, particularly in biomedical and legal domains with specialised vocabulary. This motivates domain-adaptive fine-tuning of retrieval models (Section 9.1's Legal BERT, Section 9.2's BioBERT) for production applications — a strong general-purpose model like E5 may underperform a domain-adapted model by 10–20 nDCG points on specialised retrieval tasks, even when the general model outperforms it on general web retrieval. The practical implication for Gudsky Foundation projects: always evaluate a candidate retrieval model on a sample of your target domain before deploying it in production, and consider domain-adaptive fine-tuning if the zero-shot performance is insufficient.

10.4 Adversarial Robustness of Semantic Similarity Systems

Embedding-based similarity systems are vulnerable to adversarial inputs: carefully crafted text that is semantically very different from a query but embeds nearby in the vector space, exploiting the distributional nature of the representations. BEIR benchmark analysis has shown that even strong E5 and BGE models exhibit recall degradations of 15–25% when evaluated on out-of-distribution domains compared to their in-distribution performance — a form of semantic distribution shift that behaves similarly to adversarial input.

More targeted adversarial attacks on embedding-based retrieval have been demonstrated: 'corpus poisoning' (Zhong et al., 2023) showed that inserting a small number of carefully crafted adversarial documents into a FAISS index can cause a retrieval system to consistently surface those documents in the top-K results for a wide range of queries, regardless of their actual relevance. This has direct security implications for any production RAG system that indexes documents from untrusted sources.

Mitigations include: adversarial training of the embedding model on hard negatives designed to probe specific failure modes; content filtering of indexed documents before embedding; and combining dense retrieval with BM25 (hybrid search, Section 8.4) so that purely distributional adversarial documents must also pass a keyword-matching filter. Awareness of these vulnerabilities is part of responsible deployment of semantic search, particularly in security-sensitive applications like legal, financial, or healthcare document retrieval.

10.5 Summary: Semantic Similarity Methods Landscape

Method	Context-Aware	Sentence-Level	Training Required	Best Application
WordNet (WUP/Lin)	No	Via aggregation	No	Interpretable word-level similarity; low-resource
WMD (GloVe/word2vec)	Partial	Yes (transport)	Embeddings only	Short text comparison; no GPU
TF-IDF cosine	No	Yes	No (corpus stats)	Keyword-based baseline
SBERT cosine	Yes (BERT)	Yes	Yes (fine-tuning)	Semantic search, deduplication, STS
SimCSE / E5 / BGE	Yes (contrastive)	Yes	Yes	State-of-the-art retrieval and STS
NLI entailment	Yes (full attention)	Yes	Yes	Directional inference; zero-shot classification
Cross-encoder	Yes (joint attention)	Yes (pairwise)	Yes	High-precision re-ranking

A Assignment: Semantic Search & Text Similarity Pipeline

★ GRADED ASSIGNMENT BRIEF

Build an end-to-end semantic similarity and search pipeline. Implement a bi-encoder retrieval system, evaluate on STS-B, and build a paraphrase detection classifier. Stretch goal: add a cross-encoder re-ranker and compare pipeline performance. Use the techniques from this module throughout.

A.1 Task Specifications

- **Task 1 — STS Evaluation:** Load the STS-B test set. Compute sentence embeddings using at least two models (e.g., GloVe average and SBERT all-MiniLM-L6-v2). Report Spearman correlation with human scores for both. Compare and discuss the gap.
- **Task 2 — Paraphrase Detection:** Fine-tune a BERT-based model on the MRPC training set. Evaluate on the MRPC test set reporting accuracy, F1, and a confusion matrix. Discuss false positives and false negatives.
- **Task 3 — Semantic Search Pipeline:** Build a bi-encoder retrieval system over a corpus of your choice (e.g., 10,000 Wikipedia paragraphs, Amazon product descriptions, or a domain-specific dataset relevant to your research interests). Given 5 natural-language queries, retrieve and display the top-5 results with similarity scores.
- **Stretch Goal — Re-Ranking:** Add a cross-encoder re-ranker to your Task 3 pipeline (Section 8.2). Compare precision@5 of bi-encoder alone vs. bi-encoder + cross-encoder. Discuss the latency/quality trade-off.

A.2 Grading Rubric

Component	Weight	What's Assessed
STS Evaluation (Task 1)	20%	Correct Spearman computation; insightful model comparison
Paraphrase Detection (Task 2)	25%	Correct fine-tuning; full metrics (accuracy, F1, confusion matrix); error analysis
Semantic Search (Task 3)	30%	Working retrieval pipeline; sensible query-result pairs displayed
Code Quality & Write-up	15%	Clean, documented code; 500–1000 word reflection on results and failure cases
Stretch Goal (bonus)	10%	Working re-ranker; precision@5 and latency comparison

A.3 Implementation Tips

- **Start with SBERT:** sentence-transformers makes Task 1 and Task 3 achievable in under 30 lines of Python. Get those working before investing time in Task 2's fine-tuning loop.

- **Load MRPC via Hugging Face datasets:** `load_dataset('glue', 'mrpc')` returns the standard train/validation/test splits with correct label format for Task 2.
- **FAISS for Task 3:** `pip install faiss-cpu` handles efficient nearest-neighbour search over up to a few million embeddings on a laptop CPU. See the code in Section 7.3 for a minimal working pipeline.
- **Spearman, not Pearson:** Task 1 must report Spearman rank correlation using `scipy.stats.spearmanr` — this is the standard for STS evaluation and is what published baselines report.
- **Connect STS-B loading to Task 1:** See the code in Section 5.4 for a complete STS-B evaluation loop using `load_dataset('glue', 'stsb')`.

A.4 Common Pitfalls

- **Using Pearson instead of Spearman:** Published STS-B baselines report Spearman ρ . Reporting Pearson makes your results incomparable to the literature.
- **Not unit-normalising before cosine:** If your embedding model does not normalise by default, explicitly normalise with `normalize_embeddings=True` in `sentence-transformers` or `F.normalize(emb, dim=1)` in PyTorch.
- **Evaluating on MRPC validation instead of test:** GLUE's MRPC test set has gold labels available — always report test-set numbers, not validation, for the final submission.
- **Skipping error analysis:** The write-up is 15% of the grade. A brief discussion of 3–5 false positives and 3–5 false negatives (examining the actual sentence pairs) is far more informative than aggregate metrics alone.

A.5 Submission Checklist

- STS-B evaluation: Spearman ρ reported for two or more models on the test set
- MRPC fine-tuning: accuracy, F1, and confusion matrix reported on the test set
- Semantic search: working pipeline demonstrated on 5+ queries with top-5 results and scores
- Code documented with a README covering dependencies and how to run each task
- Written reflection (500–1000 words) comparing method performance and discussing failure cases
- (Stretch) Cross-encoder re-ranker implemented; `precision@5` and latency comparison included

The future of AI is bright, and you can be part of it!